

Reviewer Pack — Immanant Log-Variance Separates Random Matrices from Det-Pad...

Entry 88defc39f508 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 19:47:04 UTC

Immanant Log-Variance Separates Random Matrices from Det-Padded Blocks

Entry ID: 88defc39f508 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 19:47:04 UTC

1. Conjecture

Field A (mathematical branch): Symmetric-group representation theory: the Specht-character immanant $\text{imm_lambda}(M) = \sum_{\sigma} \chi^{\text{lambda}(\sigma)} \prod_i M[i, \sigma(i)]$ as a partition-indexed structural decomposition of a matrix function (Stembridge 1991; Stanley EC2 Ch.7; Greene’s immanant-positivity program). The invariant is RELATIONAL (vector-valued across $p(n)$ partitions, not a single-number truth-table functional) and distinct from the blacklisted positivity-width attempt: the present invariant is the LOG-MAGNITUDE VARIANCE across the full Specht spectrum on PADDING-vs-RANDOM ensembles, an $\exp(n)$ -time meta-complexity-style discriminator, not a natural-proof-style poly-time invariant. **Field B** (complexity object): Meta-complexity in the GCT_DET_PERM regime: distinguishing K^t -random Boolean matrices from $\text{low-}K^t / \text{orbit-closure-of-det_m}^{\{\text{padded}\}}$ matrices via a super-polynomial structural test, in the spirit of Hirahara’s GapMCSP and the Mulmuley-Sohoni orbit-closure separation of perm_n from $\text{det_m}^{\{\text{poly}\}}$. The invariant $V(M)$ is a candidate meta-complexity measure that lower-bounds compressibility under linear/block padding without being itself in P.

Statement:

For M in $\{0,1\}^{\{n \times n\}}$ define $I_{\text{lambda}}(M) := \sum_{\sigma \in S_n} \chi^{\text{lambda}(\sigma)} \prod_i M[i, \sigma(i)]$ using the Murnaghan-Nakayama character table, and set $V(M) := \text{Var}_{\{\text{lambda} \in P(n)\}} (\log_2(1 + |I_{\text{lambda}}(M)|))$. Conjecture: for each n in $\{5,6,7\}$ there exist absolute constants $c_1 \geq 4c_2 > 0$ such that (i) over uniform M in $\{0,1\}^{\{n \times n\}}$, $V(M) \geq c_1(\log_2 n)^2$ on ≥ 27 of 30 seeds, while (ii) over the det-padded ensemble $M_{\text{pad}} = \text{block-diag}(M', J_{\{n-m, n-m\}})$ with $m = \text{ceil}(n/2)$, M' uniform in $\{0,1\}^{\{m \times m\}}$ and J all-ones, $V(M_{\text{pad}}) \leq c_2(\log_2 n)^2$ on ≥ 27 of 30 seeds. A single (n, seed) pair where $V(\text{uniform}) < 4V(\text{padded})$ on independent draws falsifies the conjecture.

Rationale (proposer’s reasoning):

Padding a smaller block by an all-ones submatrix forces the immanant sum to factor through the permanent of the J -block, collapsing the Specht spectrum onto partitions adjacent to (n) and (1^n) and zeroing most middle-lambda components by row/column character cancellation. Uniform random Boolean matrices instead distribute i.i.d. signed contributions across all $p(n)$ Specht coordinates, giving $\Theta(\sqrt{n!})$ magnitudes per lambda and high log-variance. The bridge from partition-indexed Specht variance to the Mulmuley-Sohoni padding question parallels how stabilizer-codimension invariants

stratify det vs perm: padding by structureless blocks erases the representation-theoretic spread that random matrices exhibit.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 253acf5b38006259

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n in $\{5, 6, 7\}$, across 30 fixed seeds, compute $V(\text{uniform})$ and $V(\text{padded})$. SUPPORTED iff at every n , the count of seeds with $V(\text{uniform}) \geq 4V(\text{padded})$ is $\geq 27/30$, AND mean $V(\text{uniform}) \geq 4\text{mean } V(\text{padded})$. FALSIFIED if any n has $< 27/30$ seeds satisfying the ratio, or any seed yields $V(\text{uniform}) < 4V(\text{padded})$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.78	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - immanant Specht character random matrix variance ensemble - geometric complexity theory determinant padding orbit closure permanent invariant - meta-complexity matrix immanant Murnaghan-Nakayama lower bound compressibility

Top relevant hits considered: - [http://arxiv.org/abs/1005.0979v1] Supersymmetry in Random Matrix Theory - [http://arxiv.org/abs/0911.5645v1] Non-Hermitian Random Matrix Ensembles - [http://arxiv.org/abs/0904.1432v2] Applications of random matrix theory to condensed matter and optical physics - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1911.03990v1] Implementing geometric complexity theory: On the separation of orbit closures via symmetries - [http://arxiv.org/abs/1306.1810v6] Matrix orbit closures

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=6.8s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict
```

```

def cycle_type(perm):
    cycles = []
    visited = [False] * len(perm)
    for i in range(len(perm)):
        if not visited[i]:
            cycle = []
            j = i
            while not visited[j]:
                visited[j] = True
                cycle.append(j)
                j = perm[j]
            cycles.append(tuple(sorted(cycle)))
    return tuple(sorted(cycles))

def murnaghan_nakayama(n):
    if n == 0:
        return {(): 1}
    if n == 1:
        return {(1,): 1}

    prev = murnaghan_nakayama(n - 1)
    current = defaultdict(int)

    for partition, value in prev.items():
        for i in range(len(partition)):
            new_partition = list(partition)
            new_partition[i] += 1
            new_partition = tuple(sorted(new_partition, reverse=True))
            current[new_partition] += value

        if len(partition) < n:
            new_partition = list(partition) + [1]
            new_partition = tuple(sorted(new_partition, reverse=True))
            current[new_partition] += value

    return dict(current)

def immanant_lambda(M, lambda_partition):
    n = len(M)
    chi = murnaghan_nakayama(n)
    if lambda_partition not in chi:
        return 0

    total = 0
    for perm in itertools.permutations(range(n)):
        ct = cycle_type(perm)
        if ct == lambda_partition:
            product = 1
            for i in range(n):
                product *= M[i][perm[i]]
            total += product * chi[lambda_partition]

    return total

def compute_variance(M):

```

```

n = len(M)
chi = murnaghan_nakayama(n)
lambda_partitions = list(chi.keys())

immanants = []
for lambda_partition in lambda_partitions:
    I = immanant_lambda(M, lambda_partition)
    if I != 0:
        immanants.append(math.log2(1 + abs(I)))

if not immanants:
    return 0.0

mean = sum(immanants) / len(immanants)
variance = sum((x - mean) ** 2 for x in immanants) / len(immanants)
return variance

def generate_random_matrix(n, seed):
    random.seed(seed)
    return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

def generate_padded_matrix(n, seed):
    random.seed(seed)
    m = (n + 1) // 2
    M_prime = [[random.randint(0, 1) for _ in range(m)] for _ in range(m)]
    J = [[1 for _ in range(n - m)] for _ in range(n - m)]

    M_pad = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(m):
        for j in range(m):
            M_pad[i][j] = M_prime[i][j]

    for i in range(n - m):
        for j in range(n - m):
            M_pad[m + i][m + j] = J[i][j]

    return M_pad

def run_trial(seed):
    n_values = [5, 6, 7]
    metric_values = []
    conjecture_holds_list = []
    counterexamples = []

    for n in n_values:
        M = generate_random_matrix(n, seed)
        M_pad = generate_padded_matrix(n, seed)

        V_uniform = compute_variance(M)
        V_padded = compute_variance(M_pad)

        metric_values.append((V_uniform, V_padded))
        conjecture_holds = V_uniform >= 4 * V_padded
        conjecture_holds_list.append(conjecture_holds)

    if not conjecture_holds:

```

```

        counterexamples.append(f"n={n}, seed={seed}, V_uniform={V_uniform}, V_padded={V_padded}")

    metric_value = sum(V_uniform for V_uniform, _ in metric_values) / len(metric_values)
    instances_tested = len(n_values)
    conjecture_holds = all(conjecture_holds_list)
    counterexample = "\n".join(counterexamples) if counterexamples else ""

    return {
        "metric_name": "log_variance_ratio",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))

    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_metric:.4f} std={std_metric:.4f} support_fraction={support_fraction:.4f}")
    else:
        counterexamples = [r["counterexample"] for r in results if r["counterexample"]]
        if counterexamples:
            first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample=\"{counterexamples[0]}\" first_failing_seed={first_failing_seed}")
        else:
            print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture_holds': True,

```

```

TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture
TRIAL: {'metric_name': 'log_variance_ratio', 'metric_value': 0.0, 'instances_tested': 3, 'conjecture
RESULT: SUPPORTED mean=0.0000 std=0.0000 support_fraction=1.0000

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The stdout shows `metric_value=0.0` across every seed with `std=0.0`, which is a strong red flag that the test is not actually computing the conjectured quantity ($V(\text{uniform})$ vs $V(\text{padded})$) — it appears to be reporting a degenerate ‘`log_variance_ratio`’ that is identically zero, not the two-sided inequality $V_{\text{uniform}} \geq c_1(\log n)^2$ AND $V_{\text{padded}} \leq c_2(\log n)^2$. Also only n in $\{5,6,7\}$ is tested with just 3 instances per seed (not 30 as the conjecture demands), so the ‘ ≥ 27 of 30 seeds’ clause is never

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Every trial reports `metric_value=0.0` with `std=0.0` and only 3 instances per seed rather than the pre-registered 30, so the two-sided inequality ($V_{\text{uniform}} \geq c_1(\log n)^2$ and $V_{\text{padded}} \leq c_2(\log n)^2$) is never actually evaluated. The degenerate identically-zero ‘`log_variance_ratio`’ indicates the test does not compute the conjectured quantity, so SUPPORTED cannot be trusted. | next: Reimplement the test to explicitly compute $V(\text{uniform})$ and $V(\text{padded})$ per seed via the Murnaghan-Nakayama character χ

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	302176
2	preregistration	claude_max	opus	0	0	5770
3	novelty	claude_max	opus	0	0	5823
4	novelty	claude_max	opus	0	0	13839
5	test_gen	mistral	codestral-latest	0	0	311895
6	test_gen	mistral	codestral-latest	0	0	316473
7	test_gen	mistral	codestral-latest	0	0	323139
8	test_gen	mistral	codestral-latest	0	0	321981
9	critic	claude_max	opus	0	0	8739
10	judge	claude_max	opus	0	0	5537

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1615373 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/88defc39f508.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/88defc39f508.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/88defc39f508.tar.gz` (if generated)